

Full Length Article

CALM: Fine-grained federated unlearning via CLIP-assisted LoRA-MoE memory editing[☆]Mang Su^a, Yang Bai^b, Caiyun Liu^c, Qihao Dong^d, Hongsheng Hu^e, Yansong Gao^f, Anmin Fu^{a,*}^a School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, Jiangsu, 210094, China^b School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, 211189, China^c China Industrial Control Systems Cyber Emergency Response Team, 35 Lugu Rd, Shijingshan District, 100040, Beijing, China^d School of Information Technology and Artificial Intelligence, Zhejiang University of Finance and Economics, Hangzhou, 310018, China^e The University of Newcastle, Newcastle, Australia^f Department of Computer Science and Software Engineering, University of Western Australia, Perth, Australia

ARTICLE INFO

Keywords:

Federated unlearning

Privacy-preserve

Memory editing

Parameter-efficient tuning

Semantic regularization

Security remediation

ABSTRACT

Federated learning (FL) enables decentralized clients to collaboratively train a global model without sharing raw data, inherently improving data confidentiality during training. With regulatory frameworks such as the General Data Protection Regulation (GDPR) mandating the *right to be forgotten*, Federated Unlearning (FU) has emerged as a key technique for removing data-related influence from trained models upon request. Existing FU methods, however, predominantly rely on costly full-model retraining or coarse parameter rollback, often leading to substantial overhead, utility degradation, and incomplete forgetting, especially under heterogeneous client distributions. To address these challenges, we propose CLIP-Assisted LoRA-MoE Memory Editing (CALM), a fine-grained FU framework that integrates Contrastive Language-Image Pretraining (CLIP), Low-Rank Adaptation (LoRA), and Mixture-of-Experts (MoE). Specifically, CALM injects modular LoRA-MoE adapters into a frozen global model, enabling targeted knowledge isolation and editing without full-model retraining. In addition, CLIP-based cross-modal regularization mitigates semantic drift during unlearning by aligning model representations with high-level semantic supervision. Experiments on CIFAR10, CIFAR100, and Caltech101 show that CALM achieves effective forgetting while keeping retained accuracy within 2–5 percentage points of retraining across the main unlearning settings; on CIFAR10-ResNet18 client-level unlearning, it obtains 66% retained accuracy (4 percentage points below retraining) and 3% forgetting accuracy, while reducing communication cost by 90.8% compared with retraining.

1. Introduction

Federated Learning (FL) (Gao et al., 2020; McMahan et al., 2017) is a distributed learning framework that allows multiple clients to collaboratively train a global model without sharing raw data, thereby enhancing resident data privacy. FL models are required not only to continuously acquire new knowledge but also to comply with the privacy regulation of “right to be forgotten” (Shastri et al., 2019) enforced by privacy acts, including GDPR (Li et al., 2019). This right ensures that users can revoke their disclosed data and request data-related information removal from trained models.

Beyond regulatory considerations, FL models are inherently vulnerable to poisoning attacks due to the limited control over participating

clients. Such attacks can significantly compromise FL systems by introducing manipulated or corrupted data during training. As a result, the global model may experience substantial accuracy degradation (Baruch et al., 2019; Fang et al., 2020; Shejwalkar & Houmansadr, 2021; Shejwalkar et al., 2022), exhibit targeted misclassifications, or embed covert backdoors that are activated by specific trigger inputs (Bagdasaryan et al., 2020; Gu et al., 2019; Xie et al., 2019). Although a range of preventive defense techniques has been proposed, achieving complete protection remains an open challenge. As an alternative, *post-hoc remediation* through FU has emerged as a promising alternative (Liu et al., 2021), enabling retrospective mitigation of security threats by precisely erasing harmful or undesired knowledge—particularly through the removal of identified malicious clients (Zhang et al., 2022b).

[☆] This work is supported by National Natural Science Foundation of China (62372236) and funded by Open Foundation of Key Laboratory of Data Protection and Security Assessment in the Industrial Field, Ministry of Industry and Information Technology (IDPSA2025-002).

^{*} Corresponding author.

E-mail address: fuam@njust.edu.cn (A. Fu).

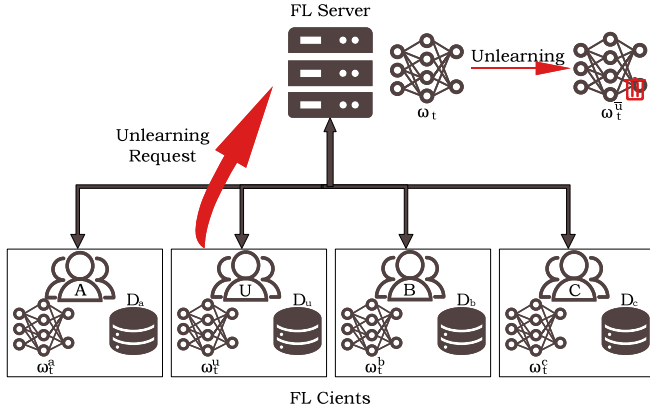


Fig. 1. FU overview. When client u requests to be forgotten, the FL server erases its contributions from the global model w_t , generating a new version w_t^δ .

Overall, FU aims to selectively remove harmful, outdated, or sensitive knowledge from collaboratively trained models, thereby ensuring compliance with privacy regulations while maintaining overall model utility (Liu et al., 2024). FU typically takes as input a trained global model and the specific data contributions requested for removal. The outcome of the unlearning process is an updated global model whose behavior closely approximates that of a model trained without incorporating the specified data. FU supports unlearning at various levels of granularity, including client-level (Liu et al., 2021), class-level (Gu et al., 2024; Wu et al., 2022b), and sample-level (Golatkari et al., 2020; Guo et al., 2019; Izzo et al., 2021; Warnecke et al., 2021) forgetting. An overview of the FU framework is presented in Fig. 1, which illustrates the removal of a client's contribution from a trained model in response to a client-level unlearning request.

Despite great efforts made, existing FL studies face three critical limitations: (1) notable performance degradation or high retraining costs (Liu et al., 2022) due to full-model retraining; (2) lack of modularity in isolating and removing specific knowledge, leading to knowledge forgetting of unrequested data; and (3) instability in non-IID environments, where unlearning disrupts semantic consistency. Semantic consistency is the ability of the model to retain coherent and meaningful relationships between data representations even when specific data points are removed. In non-IID settings, unlearning a particular subset of data can weaken or damage the model to consistently interpret and relate other, unaffected data, leading to a drift in the learned representations and subsequent accuracy drops.

To address these limitations, we propose CALM—a FU framework based on CLIP-assisted semantic regularization and LoRA-MoE memory editing. The key insights underlying our design are three-fold: (1) leveraging lightweight LoRA adapters enables efficient fine-tuning and significantly reduces the computational cost compared to full-model retraining; (2) specific knowledge can be localized within modular low-rank expert subspaces, allowing for precise and isolated removal without affecting unrelated parameters; and (3) semantic drift under non-IID conditions can be mitigated by enforcing cross-modal alignment between model representations and CLIP embeddings, thus preserving semantic consistency during unlearning.

Motivated by above insights, CALM decomposes FU into two sequential phases: first, it incorporates LoRA-MoE adapters with the frozen global model, isolating and editing client-specific knowledge; second, it employs CLIP-based semantic regularization to ensure stable adaptation and minimize unintended forgetting.

Our main contributions are summarized as follows:

- We propose CALM, a fine-grained FU framework that constructively combines existing modular LoRA-MoE adapters with CLIP-assisted semantic regularization for targeted knowledge editing under heterogeneous FL settings.

- We introduce an activation-driven expert attribution and reset strategy that localizes unlearning to a small subset of low-rank experts, thereby reducing collateral impact on unrelated knowledge compared with coarse whole-model updates.
- We conduct comprehensive experiments showing that CALM achieves strong forgetting–utility trade-offs and substantial communication savings, while also serving as an effective post-hoc remediation mechanism for backdoored FL models.

2. Preliminary and related work

This section first introduces foundational techniques such as CLIP, LoRA, and Mixture-of-Experts (MoE) that underpin recent advances in efficient and fine-grained unlearning. Then we review related work on machine unlearning (MU) and FU, highlighting key methodologies, categorization, and limitations in both centralized and federated settings.

2.1. Preliminaries on CLIP, LoRA, and MoE

CLIP (Contrastive Language-Image Pretraining) (Radford et al., 2021) is a multimodal representation learning framework trained on large-scale image-text pairs. It employs contrastive learning to align visual and textual embeddings into a shared semantic space, enabling powerful generalization capabilities for downstream vision-language tasks without task-specific fine-tuning.

LoRA (Low-Rank Adaptation) (Hu et al., 2022) is an efficient parameter adaptation approach for deep neural networks. Instead of fully fine-tuning a pretrained model, LoRA injects low-rank decomposition matrices into existing layers, significantly reducing trainable parameters while preserving the original model's generalization ability and performance.

MoE (Mixture-of-Experts) (Fedus et al., 2022; Shazeer et al., 2017) is an architectural paradigm in which multiple expert subnetworks are trained simultaneously, and a gating mechanism adaptively routes inputs to specific experts. This approach enhances model specialization, scalability, and computational efficiency by leveraging conditional computation and expert modularity.

2.2. Machine unlearning

Existing MU studies are focused on centralized learning settings (Chen et al., 2024; Liu et al., 2025). MU aims to remove the influence of specific data points from a trained model, a capability that is increasingly critical to ensuring privacy compliance, particularly under regulations of “right to be forgotten” stipulated enforced by GDPR (Li et al., 2019). Existing MU approaches can be broadly classified into two categories: data manipulation and model manipulation (Nguyen et al., 2022).

Data manipulation techniques, including data mixing and data partitioning, modify the training dataset to minimize the retained impact of data to be forgotten. Data mixing methods introduce perturbation or replace target data with interference samples (Graves et al., 2021; Tarun et al., 2023; Zhang et al., 2022a), while data partitioning divides the dataset into subsets and trains only those containing the remaining data (Bourtole et al., 2021; Chen et al., 2022a,b). Although effective, these techniques often incur significant computational overhead, especially when applied to large-scale datasets or complex models, as they still require retraining.

In contrast, model manipulation methods (Chowdhury et al., 2024; Li et al., 2024) attempt to directly alter the model to eliminate the influence of forgotten data. Such methods include model transformation, which adjusts parameters to cancel the learned effects of target samples (Golatkari et al., 2020; Izzo et al., 2021; Warnecke et al., 2021); model pruning, which removes parameters associated with the forgotten data (Baumhauer et al., 2022; Liu et al., 2021; Wang et al., 2022); and

		Sample Index					
		1	2	3	4	...	n
Client Index	A	L1	L2	L2	L3	L1	L2
	B	L2	L2	L3	L3	L3	L2
	C	L1	L1	L2	L3	L4	L4
	D	L3	L2	L1	L1	L4	L4
	...	L3	L3	L3	L3	L4	L4
	n	L2	L1	L1	L2	L4	L4

Client
Unlearning

Class
Unlearning

Sample
Unlearning

Fig. 2. Visualization of the three levels of FU: *client unlearning*, *class unlearning*, and *sample unlearning*. Each cell represents a sample held by a specific client, with the label (L1–L4) denoting the class of that sample. Rows correspond to different clients (A to n), and columns correspond to sample indices within each client's local dataset. In this example: (1) *Client-level unlearning* (yellow box highlight) removes all data from a particular client (e.g., client A); (2) *Class-level unlearning* (red box highlight) removes all samples belonging to a particular class (e.g., all L4-labeled samples from all clients); (3) *Sample-level unlearning* (blue box highlight) removes a specific data sample (e.g., samples with label L3 and L2 from client D).

model replacement, which swaps out affected components with precomputed submodels. Although these approaches are generally more computationally efficient than full retraining, they often require complex interventions in the model architecture and may not fully guarantee the elimination of all residual influences, particularly when the forgotten data have been deeply entangled within the model's internal representations.

2.3. Federated unlearning

Distinct from centralized machine learning, FU (Liu et al., 2020, 2024) introduces a broader and more nuanced perspective on unlearning, encompassing not only client-level (Liu et al., 2021) unlearning but also class-level (Gu et al., 2024; Wu et al., 2022b) and sample-level unlearning (Golatkhar et al., 2020; Guo et al., 2019; Warnecke et al., 2021). Fig. 2 provides an illustrative overview of different levels of FU.

In this context, existing FU methods can generally be grouped into two main categories: retraining-based approaches (Liu et al., 2022) and parameter manipulation-based approaches (Cao et al., 2023; Che et al., 2023; Halimi et al., 2022; Wang et al., 2023). Retraining-based approaches retrain a new global model from scratch while excluding the data or clients that need to be forgotten. For instance, when a particular client's data must be removed, the approach proposed in Liu et al. (2021) retrains the remaining clients to compute corrected gradient directions, which are then aggregated by the server to update the global model. In Liu et al. (2022), a quasi-Newton optimization method is introduced to accelerate the retraining process. Additionally, Liu et al. (2020) reduce retraining overhead by employing clustering techniques to approximate the forgetting process. Despite these efforts, retraining-based strategies remain computationally expensive and often impractical for large-scale, real-world deployments.

To address these challenges, parameter manipulation-based methods have been explored as more efficient alternatives. Wang et al. (2022) propose a pruning-based approach for CNNs, removing class-related channel parameters to achieve class-level unlearning. In another line of work, Wu et al. (2022a) aim to erase a specific client's contribu-

tion by subtracting its historical updates from the global model. The previous version of the global model is then used as a teacher in a knowledge distillation framework to recover performance on retained data while ensuring that the forgotten client's influence is effectively removed.

Nonetheless, FU is still limited, with most existing works primarily focusing on client-level unlearning. Moreover, challenges such as knowledge interference and the irreversibility of the unlearning process have not been adequately addressed. Furthermore, current FU approaches are incapable of achieving fine-grained unlearning, such as selectively removing specific samples or classes (i.e., sample-level or class-level unlearning), and exhibit suboptimal performance when applied to heterogeneous data, where forgetting does not scale effectively across imbalanced classes.

3. Problem statement

This section formalizes the problem of federated unlearning (FU) and its unique challenges. We first formulate centralized machine unlearning (MU) as a preliminary, and then present the formal definitions of FU, with a focus on its three levels: client-level, class-level, and sample-level unlearning. We also describe the threat model considered in this work, which specifies the attacker's black-box membership inference capability and the defender's post-hoc unlearning role in federated systems.

3.1. Centralized machine unlearning

In centralized machine learning, MU seeks to remove the influence of a subset of training data D^u from a trained model. Given the full dataset $D = \{(x_i, y_i)\}_{i=1}^n$, with $D' = D \setminus D^u$ as the retained set, the objective is to obtain a model $\mathcal{M}^f$ such that:

$$\mathcal{M}^f = \text{FGT}(\mathcal{M}^r, D', D^u), \quad (1)$$

where $\text{FGT}(\cdot)$ is the unlearning for forget operation. The goals are to maximize forgetting on D^u , preserve utility on D' , and minimize computational cost. Denoting $F(\cdot)$ as the model test loss and $\text{RC}(\cdot)$ as resource consumption, the above objectives can be respectively expressed as:

$$\max F(\mathcal{M}^f, D^u), \quad (2)$$

$$\min F(\mathcal{M}^f, D'), \quad (3)$$

$$\min \text{RC}(\text{FGT}(\mathcal{M}^r, D', D^u)). \quad (4)$$

Retraining from scratch on D' provides an ideal baseline but is often impractical. These challenges become even more significant in federated settings due to data decentralization, privacy, and communication constraints.

3.2. Federated unlearning

FU extends MU to federated learning, where data is distributed across N clients and direct data aggregation is prohibited. The federated dataset is $D = \bigcup_{k=1}^N D_k$, where each D_k is the private data held by client k . FU seeks to remove the influence of a target unlearning set $D^u \subset D$, while maximizing utility and efficiency on the retained set $D' = D \setminus D^u$. There are three main unlearning scenarios in FL (Golatkhar et al., 2020; Gu et al., 2024; Liu et al., 2021; Wu et al., 2022b).

3.2.1. Client unlearning

In client unlearning, the objective is to erase the contribution of all data from a subset of clients $\mathcal{N}_u \subseteq \mathcal{N} = \{1, 2, \dots, N\}$. The unlearning set is $D^u = \bigcup_{k \in \mathcal{N}_u} D_k$, and the retained set is $D' = \bigcup_{k \in \mathcal{N} \setminus \mathcal{N}_u} D_k$. The FU objective in this setting are:

$$\max \sum_{k \in \mathcal{N}_u} F(\mathcal{M}^f, D_k), \quad (5)$$

$$\min_{k \in \mathcal{N} \setminus \mathcal{N}_u} F(\mathcal{M}^f, \mathcal{D}_k), \quad (6)$$

$$\min \text{RC}(\text{FGT}(\mathcal{M}^f, \{\mathcal{D}_k\}_{k \in \mathcal{N}})). \quad (7)$$

3.2.2. Class unlearning

Class unlearning aims to remove all knowledge related to one or more classes $C^u \subseteq C$, where C is the full set of class labels. The unlearning set is

$$\mathcal{D}^u = \{(x_i^k, y_i^k) \mid y_i^k \in C^u, k \in \mathcal{N}\},$$

and the retained set includes all samples with $y_i^k \notin C^u$. The optimization objectives are the same as (2)–(4).

3.2.3. Sample unlearning

Sample unlearning focuses on the removal of specific data records, potentially distributed across multiple clients. Here, $\mathcal{D}^u$ is a collection of selected samples (x_i^k, y_i^k) , and $\mathcal{D}^r$ is the complement. The objectives are the same as (2)–(4).

In all scenarios, the ideal FU method would yield a model $\mathcal{M}^f$ that performs on $\mathcal{D}^r$ and $\mathcal{D}^u$ as if it had never seen $\mathcal{D}^u$, matching retraining on the retained data—while also being practical in terms of computation, communication, and privacy.

Overall, FU aims to efficiently and privately erase the influence of specified clients, classes, or samples from the global model, while preserving utility on retained data and adhering to the efficiency and privacy constraints of federated systems. This introduces significant challenges beyond centralized unlearning, motivating the development of dedicated FU algorithms.

3.3. Threat model

We consider a threat model involving two entities: an **attacker**, an external adversary aiming to infer membership information by querying an inference API, and a **defender**, namely the FL server that maintains the global model and performs post-hoc unlearning.

Attacker. The attacker operates in a black-box setting and can only query the deployed inference API to obtain output confidence vectors. The attacker has no direct access to model parameters, intermediate activations, or clients' raw data, but knows the model architecture and may use auxiliary data to construct membership inference attacks.

Defender. The defender coordinates model aggregation and processes unlearning requests at the client, class, or sample level. The defender knows the identities of requesting clients and the specification of the unlearning target, but cannot access any client's raw local data.

Under this threat model, *CALM* is designed to reduce residual influence from the specified unlearning target while preserving utility on the retained data. In addition, when malicious clients are identified after deployment, the same mechanism can be used for post-hoc backdoor remediation.

4. The proposed method: CALM

This section introduces the proposed method, *CALM*, which enables efficient and fine-grained federated unlearning. We first present a high-level overview of the framework—summarizing its key insights and overall workflow—and then provide detailed descriptions of its main components.

4.1. CALM overview

CALM is built on three key insights: (1) Specific knowledge can be modularly attributed to low-rank adapter subspaces, supporting targeted and precise forgetting; (2) Adapter-based modular editing enables highly efficient unlearning, reducing overhead compared to retrain the entire model; (3) Semantic consistency across non-IID data is preserved

via CLIP-based cross-modal supervision, mitigating accuracy loss and collateral forgetting. The overview is shown in Fig. 3.

The *CALM* workflow comprises the following main steps:

1. **Federated learning:** All clients collaboratively train a global model using standard FL procedures, with each client keeping local data private.
2. **LoRA-MoE dynamic expert routing:** Upon receiving an unlearning request (e.g., from a specific client, class, or data subset), the server freezes the global model parameters and injects LoRA-MoE adapters into all linear layers, distributing these adapters to all clients. Each client independently employs a gating network to dynamically activate subsets of adapter experts based on the relevance of their local data only. Particularly, the requesting client locally identifies and deactivates (by masking or resetting) the experts most relevant to its own data to be forgotten, while other clients continue standard expert routing. During subsequent aggregation of LoRA-MoE adapters, the targeted knowledge from the requesting client is effectively and modularly erased without exposing private data across clients.
3. **CLIP-assisted regularization:** Each client fine-tunes its local adapters on its local retained data $\mathcal{D}_k^r$, with a CLIP-based cross-modal regularization term added to the loss. This encourages semantic consistency in vision-language representations, stabilizing the model and mitigating side effects of unlearning. Only the LoRA-MoE adapters are updated in this process, while the frozen global model parameters remain unchanged.
4. **Aggregation and model update:** Clients upload their updated adapters to the server, which aggregates them (e.g., via weighted averaging) to form new global adapters. The server redistributes the aggregated adapters to all clients for the next unlearning round. This process repeats until the unlearning objectives are met and the global model converges.

The *CALM* workflow is detailed in Algorithm 1.

4.2. LoRA-MoE dynamic expert routing

Upon receiving an unlearning request (e.g., from a specific client, class, or data subset), the server freezes the backbone parameters of the global model and integrates LoRA-MoE adapters into all linear layers. These adapters are then distributed to each client for local adaptation. To ensure privacy, each client independently performs dynamic expert routing using a gating network based exclusively on their local data. Specifically, the gating network computes relevance scores for each expert given the client's own input data, enabling each client to selectively activate the subset of experts most relevant to its local knowledge.

For the client that requests unlearning (e.g., client a), it independently conducts local attribution analysis solely on its private data to identify experts strongly associated with the knowledge it seeks to erase. The identified experts are then deactivated either by resetting their parameters or applying a mask, ensuring selective and modular removal of client-specific knowledge. Importantly, this operation is conducted privately on the requesting client's own device, with no visibility into its data by other clients.

The gating network responsible for dynamic expert routing is denoted by $G(x)$, where x represents the intermediate feature map generated by the corresponding convolutional or preceding layer. The network outputs relevance scores, activating only the Top- k most relevant experts for each local data sample. The adapted weight for input x is computed as:

$$W'(x) = W + \sum_{j=1}^M G_j(x)(A_j B_j), \quad (8)$$

where $G_j(x)$ is the gating score (typically computed via softmax or sparsemax functions) for expert j , and $A_j \in \mathbb{R}^{d \times r}$, $B_j \in \mathbb{R}^{r \times h}$ represent the low-rank matrices associated with expert j .

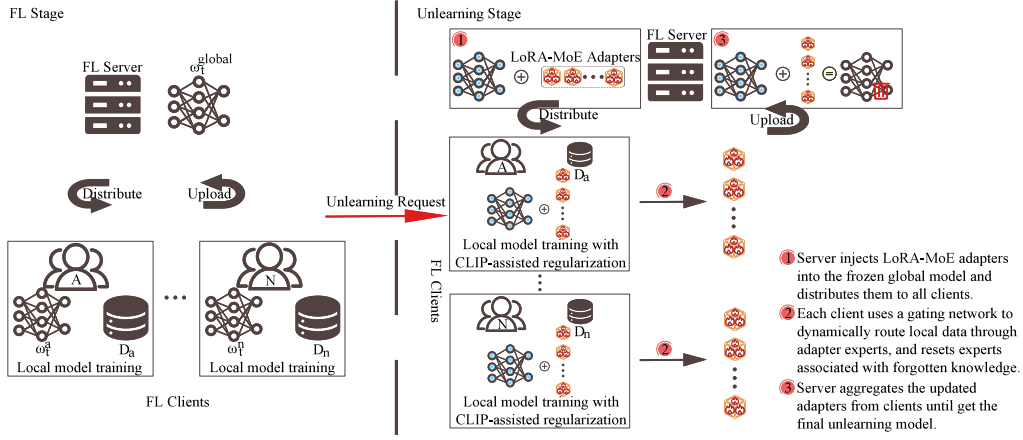


Fig. 3. Overview of CALM. (Left) FL stage: collaborative model training. (Right) Unlearning stage: after unlearning request, LoRA-MoE adapters are injected, a gating network routes inputs to knowledge-relevant experts, and CLIP-assisted regularization preserves semantics during forgetting.

Algorithm 1 CALM: CLIP-assisted LoRA-MoE memory editing

Input: Initial global model $\mathcal{M}$; client datasets $\{D_k\}$; unlearning target D^u ; retained datasets $\{D'_k\}$ with $D' = \bigcup_{k=1}^N D'_k$; CLIP encoder ϕ_{CLIP}

Parameters: LoRA-MoE adapters; gating network; loss weights λ_f , λ_c , λ_{img} , and λ_{text}

Output: Unlearned global model $\mathcal{M}^*$

```

1: Collaborative FL Training:
2: for each FL round do
3:   for each client  $k$  do
4:     Download  $\mathcal{M}$  and perform local update on  $D_k$ 
5:     Upload local update to server
6:   end for
7:   Server aggregates local updates
8: end for
9: Freeze backbone parameters
10: LoRA-MoE Dynamic Expert Routing:
11: Inject LoRA-MoE adapters and initialize gating network
12: for each input sample do
13:   Use gating network for expert selection and routing
14: end for
15: Compute attribution scores  $a_j = \frac{1}{|D^u|} \sum_{x \in D^u} G_j(x)$  for all experts
16: Select Top- $k$  experts to form the unlearning-relevant set  $\mathcal{E}^u$ 
17: for each expert  $e \in \mathcal{E}^u$  do
18:   Reset or mask parameters of  $e$ 
19: end for
20: CLIP-Assisted Regularization and Iterative Unlearning:
21: for each unlearning communication round do
22:   for each client  $k$  do
23:     Fine-tune adapters on  $D'_k$  using

```

$$\mathcal{L}_{\text{CALM}} = \mathcal{L}_{\text{task}} + \lambda_f \mathcal{L}_{\text{forget}} + \lambda_c \mathcal{L}_{\text{CLIP}}$$

```

24:   Upload adapter updates to server
25:   end for
26:   Server aggregates and redistributes updated adapters
27: end for
28: Output: Final unlearned global model  $\mathcal{M}^*$ 

```

Once an unlearning request is received (e.g., from a specific client, class, or data subset), the gating network is locally applied by each client to dynamically identify and activate subsets of experts relevant to its own data. For the target set to be forgotten (D^u), we instantiate local attribution analysis using expert-activation statistics. Specifically, for expert j , we compute its mean routing score over the target set as

$$a_j = \frac{1}{|D^u|} \sum_{x \in D^u} G_j(x). \quad (9)$$

Experts with the Top- k attribution scores are then selected as the unlearning-relevant expert set

$$\mathcal{E}^u = \text{TopK}_{j \in \{1, \dots, M\}}(a_j). \quad (10)$$

Intuitively, this operation identifies the experts that are most consistently activated by the target knowledge and are therefore most likely to encode its contribution. In the case of client-level unlearning, the re-

questing client independently identifies these experts solely based on its local dataset D^u_a , ensuring that no private data is shared or visible to other clients.

The identified experts ($\mathcal{E}^u$) are then reset (parameters re-initialized) or masked locally, while non-requesting clients continue normal expert activation based on their own retained data. Importantly, CALM does not reset all routed experts indiscriminately; instead, it only edits the Top- k experts with the highest attribution scores. This design is particularly important for sample-level unlearning, where an individual sample may share experts with many retained samples. Our design follows the same general intuition as recent localized/selective unlearning methods, which show that editing only a small subset of important parameters can provide a better unlearning-utility-efficiency trade-off than coarse full-parameter modification (Fan et al., 2023; Torkzadehmahani et al., 2024). By restricting the editing operation to the most strongly associated experts and subsequently re-optimizing the adapters on re-

tained data, *CALM* reduces collateral interference on unrelated knowledge, although such interference cannot be completely eliminated in the finest-grained setting.

During the subsequent global aggregation, the locally adapted (or reset) experts from all clients are aggregated. As a result, the global LoRA-MoE adapters reflect selective removal of the targeted knowledge while preserving the majority of unrelated expert subspaces. Therefore, for client- and class-level unlearning, this mechanism provides modular and efficient knowledge removal with limited impact on retained knowledge; for sample-level unlearning, it should be interpreted as an approximate fine-grained editing mechanism rather than an exact guarantee. This design choice explains why sample-level unlearning is inherently more challenging than client- or class-level unlearning, and also motivates the use of retained-data fine-tuning with CLIP-assisted regularization in the subsequent stage to recover semantic consistency and utility.

4.3. CLIP-assisted regularization

After the expert reset, the local model equipped with LoRA-MoE adapters on each client may suffer from semantic drift or performance loss. To address this, *CALM* integrates CLIP-assisted cross-modal regularization during the adapter fine-tuning phase. Specifically, for each retained sample $(x, y) \in D'_k$ on client k :

- $\phi_{\text{CLIP}}^{\text{img}}(x)$: embedding from CLIP image encoder,
- $\phi_{\text{CLIP}}^{\text{text}}(y)$: embedding from CLIP text encoder,
- $\phi_{\text{model}}(x)$: output embedding from the local model equipped with LoRA-MoE adapters on each client.

The regularization loss enforces semantic alignment:

$$\mathcal{L}_{\text{img}} = 1 - \cos(\phi_{\text{model}}(x), \phi_{\text{CLIP}}^{\text{img}}(x)), \quad (11)$$

$$\mathcal{L}_{\text{text}} = 1 - \cos(\phi_{\text{model}}(x), \phi_{\text{CLIP}}^{\text{text}}(y)), \quad (12)$$

$$\mathcal{L}_{\text{CLIP}} = \lambda_{\text{img}} \mathcal{L}_{\text{img}} + \lambda_{\text{text}} \mathcal{L}_{\text{text}}, \quad (13)$$

where λ_{img} and λ_{text} are balancing hyperparameters.

This cross-modal regularization not only helps recover semantic consistency after expert editing, but also reduces unintended drift on retained knowledge by anchoring the adapted representation to semantically aligned image and label embeddings. In this sense, CLIP supervision serves as an additional safeguard against excessive collateral damage during fine-grained editing.

To explicitly suppress the memorized influence of the unlearning target, we define the forgetting loss on the target set as

$$\mathcal{L}_{\text{forget}} = \frac{1}{|D^u|} \sum_{(x,y) \in D^u} \log p_{\theta}(y|x), \quad (14)$$

where $p_{\theta}(y|x)$ is the model confidence assigned to the original label. Minimizing this term discourages confident prediction on the data to be forgotten and promotes behavior closer to a model retrained without D^u .

The final unlearning objective combines utility, forgetting, and semantic regularization, expressed as:

$$\mathcal{L}_{\text{CALM}} = \mathcal{L}_{\text{task}} + \lambda_f \mathcal{L}_{\text{forget}} + \lambda_c \mathcal{L}_{\text{CLIP}}, \quad (15)$$

where $\mathcal{L}_{\text{task}}$ is the classification loss on the local retained data D'_k , and λ_f , λ_c are weighting factors.

Collaborative Adapter Fine-tuning. Clients fine-tune their LoRA-MoE adapters using the above objective on their local retained data. This collaborative process not only accelerates convergence but also helps maintain global model consistency.

4.4. Aggregation and model update

After each round of local efficient fine-tuning, clients upload their updated adapters only to the server. The server aggregates these

adapters (e.g., by averaging) and redistributes the updated adapters for the next round. This process iterates for multiple rounds, progressively improving the unlearned global model's utility and stability. The global model remains frozen throughout, and only adapter parameters are updated. This iterative aggregation ensures that forgetting is robust, model utility is preserved, and privacy risks from the forgotten data are mitigated before deployment.

5. Experiments

5.1. Setup

All experiments are implemented using PyTorch 2.2.0, and the *CALM* framework is developed based on a standard FL architecture consisting of one central server and multiple clients. Unless otherwise stated, the number of clients is set to be 50 for extensive evaluations. Experiments are conducted on a workstation equipped with an NVIDIA RTX 4090 GPU. Both SGD and Adam optimizers are adopted with a batch size of 128.

Dataset and Model. Three benchmark datasets are used in the experiments: CIFAR10 (Krizhevsky, 2009), Caltech101 (Fei-Fei et al., 2004), and CIFAR100 (Krizhevsky, 2009). CIFAR10 and CIFAR100 are widely used image-classification datasets containing 60,000 32×32 color images each; CIFAR10 covers 10 balanced classes, while CIFAR100 contains 100 fine-grained categories with only 600 images per class. Caltech101 contains approximately 9,000 images from 101 object categories and exhibits larger inter-class diversity and higher visual resolution.

We selected these datasets for three reasons. First, together they span low-, medium-, and high-class-cardinality recognition regimes, which is useful for evaluating how forgetting behaves when semantic granularity varies. Second, they are standard benchmarks in both FL and FU, allowing direct comparison with prior baselines such as FedEraser (Liu et al., 2021), Exact-Fun (Xiong et al., 2023), and EraseClient (Halimi et al., 2022). Third, although they are not privacy-sensitive domains by themselves, they provide controlled testbeds for isolating unlearning behavior before moving to application-specific domains with additional confounders.

To simulate heterogeneous (non-IID) conditions, we partition data among clients according to a Dirichlet distribution with concentration parameter $\alpha = 1.0$, following Hsu et al. (2019). A smaller concentration parameter induces more severe client heterogeneity; in particular, $\alpha = 0.3$ corresponds to a substantially more skewed setting than $\alpha = 1.0$. Unless otherwise specified, the main results are reported under $\alpha = 1.0$.

For model architectures, we employ ResNet18 on CIFAR10 and ResNet50 on CIFAR100 and Caltech101. We chose these backbones for two pragmatic reasons: (1) they remain widely used reference architectures in representative FU studies (Halimi et al., 2022; Liu et al., 2021; Xiong et al., 2023), making comparisons with prior methods fairer; and (2) their moderate capacity allows controlled measurement of forgetting, utility, time, and communication trade-offs without conflating the analysis with very large modern training recipes.

Comparative Works. We compare *CALM* against the following baselines: Retrain(upper bound), FedEraser (Liu et al., 2021), Exact-Fun (Xiong et al., 2023), and EraseClient (Halimi et al., 2022). Among them, Retrain represents the ideal case where the model is re-initialized and retrained from scratch without access to the unlearned data. FedEraser (Liu et al., 2021) is an early dedicated federated unlearning method that reconstructs an unlearned model by leveraging historical client updates retained at the server and employing a calibration strategy to accelerate model reconstruction. Exact-Fun (Xiong et al., 2023) develops an exact and efficient unlearning mechanism tailored for FL by combining quantized model updates with a deletion algorithm. EraseClient (Halimi et al., 2022) targets client-level unlearning by performing local unlearning on the target client, followed by a few communication rounds with the remaining clients to refine the global model.

We select these methods because they span retraining-based, update-calibration, and local-deletion paradigms and remain the most directly comparable baselines for our setting. Recent FU studies published after these methods have explored additional settings and stronger assumptions, but many target vertical FL, differential privacy, graph data, or other scenarios that are not directly comparable to our horizontal image-classification setting.

Metrics. We evaluate *CALM* using below metrics:

- RA (Retained Accuracy) refers to the testing accuracy on the retained dataset D' , which should remain as high as possible to minimize knowledge interference.
- FA (Forgetting Accuracy) is the testing accuracy on the unlearned dataset D'' , and a lower value indicates better effective forgetting.
- MIA (Membership Inference Accuracy) measures privacy leakage by evaluating whether an attacker can distinguish training members from non-members. We adopt a black-box setting in which the attacker only observes the model's output confidence vector. Following the standard shadow-model paradigm, we train 4 shadow models with the same backbone as the target model on data subsets that are sampled from the same distribution as the target task but do not overlap with the victim model's training data. For each shadow model, the confidence vectors on its own training samples are used as member attack data, while the confidence vectors on a disjoint hold-out set are used as non-member attack data. A two-hidden-layer MLP attack classifier ($C \rightarrow 128 \rightarrow 64 \rightarrow 1$) is then trained using these confidence vectors as input and a binary membership label (member vs. non-member) as output, with Adam (learning rate 10^{-3} , batch size 128, up to 50 epochs, early stopping patience = 5). Attack accuracy is finally reported on a held-out attack-test split. An ideal post-unlearning MIA accuracy is close to 0.5; we therefore interpret MIA as a relative privacy-risk indicator rather than a formal privacy guarantee.
- ReA (Relearning Accuracy) measures the performance when the forgotten data is reintroduced and retrained over a fixed number of epochs. A lower ReA suggests that the knowledge was effectively erased and is harder to relearn.
- Time measures the total wall-clock time required to perform unlearning, including all rounds of computation and communication, under the given experimental protocol.
- Comm (Communication) measures the average volume of data transmitted between a client and the server during the unlearning process, reflecting the overall bandwidth cost. Unless otherwise specified, each experiment is repeated five times with different random seeds, and the reported RA, FA, ReA, MIA, and Time values are presented as mean $\pm$ standard deviation. Since communication volume is determined by the update protocol and parameter size, Comm is reported as a single deterministic value.

5.2. Results analysis

We conduct comprehensive evaluations of the proposed *CALM* method on three unlearning levels—*client-level*, *class-level*, and *sample-level*—across multiple benchmark datasets.

5.2.1. Client unlearning

Table 1 details the client unlearning performance across different datasets and model architectures. On CIFAR10 with ResNet18, *CALM* achieves a retained accuracy of 66%, which is lower than retraining (70%) and comparable to FedEraser, but higher than E-F and E-C. *CALM* attains the lowest forgetting accuracy of 3%, demonstrating the most effective removal of target client knowledge among all compared methods. The relearning accuracy is also the lowest at 42%, indicating strong resistance to knowledge recovery. In terms of privacy, *CALM* achieves a MIA accuracy of 67%, which is comparable to FedEraser and better than E-F and E-C. Although this value is not ideal (ideally close to 50% for random guessing), it is still competitive relative to the baselines.

While the time cost for *CALM* (473.22 seconds) is higher than that of E-C (197.66 seconds), *CALM* dramatically reduces the communication cost to 3.92M, representing more than a 90% reduction compared to retraining (42.73M). This increase in computation time is primarily due to the adapter-based dynamic expert routing and the CLIP-guided fine-tuning steps, which provide significant gains in communication efficiency.

For the more challenging Caltech101 dataset with ResNet50, which features a larger number of classes and increased inter-class diversity, *CALM* achieves a retained accuracy of 46%, approaching that of retraining (51%) and outperforming all other baselines. The forgetting accuracy is 2%, which is nearly as low as the retraining baseline (1%) and better than all non-retraining methods. The relearning accuracy of 15% also indicates effective resistance to knowledge recovery. Regarding privacy, *CALM* obtains a MIA accuracy of 70%, which is close to FedEraser and E-C, though not ideal. In terms of computational cost, *CALM* significantly reduces communication overhead to 8.13M, compared to 97.32M for retraining, while maintaining a moderate time cost of 632.33 seconds.

On CIFAR100 with ResNet50, *CALM* maintains strong and stable performance. It achieves a forgetting accuracy of 2%, matching the retraining approach and FedEraser, and reaches a retained accuracy of 46%, the highest among all non-retraining baselines. The relearning accuracy is 17%, and the MIA accuracy is 64%, which is the lowest among the non-retraining approaches and competitive with retraining (61%). Notably, the time cost for *CALM* is 527.18 seconds, which is lower than retraining (1394.38 seconds), E-F (1004.39 seconds), and FedEraser (974.52 seconds), and second only to E-C (324.37 seconds). Communication volume is also significantly reduced to 8.46M.

This phenomenon can be understood from the fact that, in image classification, membership inference does not depend solely on whether a specific sample has been removed from training. It can also exploit residual statistical cues in the model outputs, such as confidence calibration, class-specific fitting patterns, and the difference between training and non-training distributions. As a result, even full retraining may not completely eliminate these signals, especially when the model still fits the training distribution well. Consistent with this interpretation, we observe that MIA accuracy remains above 50% even for retraining. Therefore, MIA in our setting should be interpreted as a relative leakage indicator rather than evidence of complete privacy removal. Fig. 4 visualizes the confidence-score distributions of member and non-member samples under the CIFAR10-ResNet18 client-level setting. The retrained model shows highly overlapping confidence distributions, but a small residual confidence gap remains, which explains why its MIA accuracy is still above random guessing.

Overall, *CALM* achieves a favorable balance among retained utility, forgetting effectiveness, and resource efficiency. Across all benchmark datasets (CIFAR10, Caltech101, CIFAR100) and model architectures (ResNet18, ResNet50), *CALM* generally outperforms the other efficient non-retraining baselines, while trading moderately higher computation for substantial communication savings.

5.2.2. Class unlearning

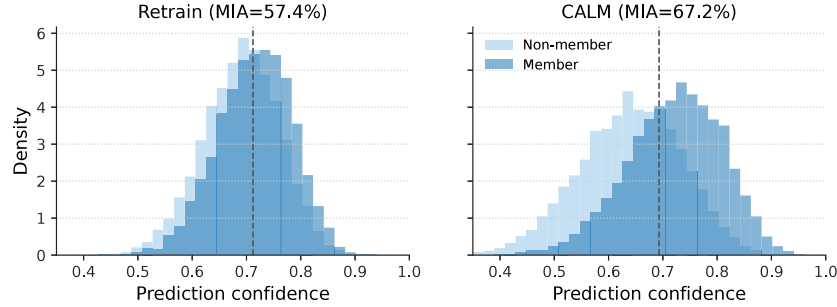
Table 2 summarizes the results of class unlearning experiments across different datasets and models.

On CIFAR10 with ResNet18, *CALM* achieves a retained accuracy of 72%, which is only slightly lower than retraining at 74%. In terms of forgetting effectiveness, *CALM* reaches a forgetting accuracy of 2%, while retraining achieves 1%. For relearning accuracy, *CALM* attains 97%, compared to 100% for retraining. The membership inference accuracy for *CALM* is 66%, higher than retraining's 56%, but still indicating a moderate level of privacy protection. Importantly, the computation time required by *CALM* is 374.57 seconds, which is less than half that of retraining (805.49 seconds), demonstrating a substantial efficiency gain. In addition, *CALM* reduces the communication cost from 42.73M to 3.92M, showing that its adapter-only update strategy preserves the same communication advantage observed in the client-level setting.

Table 1

Comparison of unlearning methods in **client unlearning** setting on different datasets and models. RA, FA, ReA, MIA, and Time are reported as mean $\pm$ standard deviation over five independent runs. Communication volume is deterministic under the given protocol and is therefore reported as a single value. $\uparrow$: higher is better; $\downarrow$: lower is better. “E-F” denotes Exact-Fun, and “E-C” denotes EraseClient.

Dataset & Model	Method	RA $\uparrow$ (%)	FA $\downarrow$ (%)	ReA $\downarrow$ (%)	MIA $\downarrow$ (%)	Time $\downarrow$ (s)	Comm $\downarrow$ (M)
CIFAR10 ResNet18	Retrain	70 $\pm$ 0.65	4 $\pm$ 0.36	50 $\pm$ 0.64	57 $\pm$ 0.83	901.56 $\pm$ 40.4	42.73
	FedEraser	66 $\pm$ 0.38	4 $\pm$ 0.57	49 $\pm$ 0.51	67 $\pm$ 0.43	684.53 $\pm$ 31.6	42.73
	E-F	64 $\pm$ 0.61	6 $\pm$ 0.43	43 $\pm$ 0.71	72 $\pm$ 0.39	814.56 $\pm$ 25.4	42.73
	E-C	63 $\pm$ 0.49	6 $\pm$ 0.51	57 $\pm$ 0.62	69 $\pm$ 0.44	197.66 $\pm$ 17.3	42.73
	CALM (Ours)	66 $\pm$ 0.39	3 $\pm$ 0.49	42 $\pm$ 0.71	67 $\pm$ 0.38	473.22 $\pm$ 21.4	3.92
Caltech101 ResNet50	Retrain	51 $\pm$ 0.42	1 $\pm$ 0.37	19 $\pm$ 0.36	58 $\pm$ 0.41	1149.64 $\pm$ 31.8	97.32
	FedEraser	37 $\pm$ 0.62	3 $\pm$ 0.28	14 $\pm$ 0.41	69 $\pm$ 0.38	893.52 $\pm$ 30.3	97.32
	E-F	40 $\pm$ 0.44	3 $\pm$ 0.22	17 $\pm$ 0.41	75 $\pm$ 0.24	946.17 $\pm$ 32.4	97.32
	E-C	42 $\pm$ 0.33	3 $\pm$ 0.35	15 $\pm$ 0.39	69 $\pm$ 0.22	283.15 $\pm$ 30.4	97.32
	CALM (Ours)	46 $\pm$ 0.37	2 $\pm$ 0.34	15 $\pm$ 0.27	70 $\pm$ 0.31	632.33 $\pm$ 31.6	8.13
CIFAR100 ResNet50	Retrain	48 $\pm$ 0.55	2 $\pm$ 0.34	18 $\pm$ 0.42	61 $\pm$ 0.37	1394.38 $\pm$ 37.4	97.32
	FedEraser	36 $\pm$ 0.31	2 $\pm$ 0.49	13 $\pm$ 0.23	67 $\pm$ 0.50	974.52 $\pm$ 34.6	97.32
	E-F	45 $\pm$ 0.41	3 $\pm$ 0.51	17 $\pm$ 0.51	74 $\pm$ 0.93	1004.39 $\pm$ 38.6	97.32
	E-C	43 $\pm$ 0.52	3 $\pm$ 0.47	21 $\pm$ 0.42	68 $\pm$ 0.35	324.37 $\pm$ 29.6	97.32
	CALM (Ours)	46 $\pm$ 0.53	2 $\pm$ 0.31	17 $\pm$ 0.33	64 $\pm$ 0.37	527.18 $\pm$ 19.2	8.46

**Fig. 4.** Confidence-score distributions of member and non-member samples under the CIFAR10-ResNet18 client-level setting.**Table 2**

Comparison of class unlearning methods on CIFAR10, Caltech101, and CIFAR100. RA, FA, ReA, MIA, and Time are reported as mean $\pm$ standard deviation over five independent runs.

Dataset&Model	Method	RA $\uparrow$ (%)	FA $\downarrow$ (%)	ReA $\downarrow$ (%)	MIA $\downarrow$ (%)	Time $\downarrow$ (s)	Comm $\downarrow$ (M)
CIFAR10 ResNet18	Retrain	74 $\pm$ 0.39	1 $\pm$ 0.33	100 $\pm$ 0.59	56 $\pm$ 0.29	805.49 $\pm$ 27.9	42.73
	CALM (Ours)	72 $\pm$ 0.49	2 $\pm$ 0.24	97 $\pm$ 0.49	66 $\pm$ 0.52	374.57 $\pm$ 13.6	3.92
Caltech101 ResNet50	Retrain	39 $\pm$ 0.33	1 $\pm$ 0.44	100 $\pm$ 0.57	66 $\pm$ 0.49	953.47 $\pm$ 24.6	97.32
	CALM (Ours)	35 $\pm$ 0.23	2 $\pm$ 0.45	96 $\pm$ 0.63	70 $\pm$ 0.41	604.72 $\pm$ 35.3	8.13
CIFAR100 ResNet50	Retrain	34 $\pm$ 0.53	1 $\pm$ 0.43	100 $\pm$ 0.66	61 $\pm$ 0.39	1140.77 $\pm$ 24.5	97.32
	CALM (Ours)	31 $\pm$ 0.35	3 $\pm$ 0.19	96 $\pm$ 0.49	69 $\pm$ 0.43	783.92 $\pm$ 18.4	8.46

For the Caltech101 dataset with ResNet50, *CALM* achieves a retained accuracy of 35%, which is somewhat lower than retraining’s 39%. The forgetting accuracy with *CALM* is 2%, compared to 1% for retraining. Relearning accuracy reaches 96% for *CALM* and 100% for retraining. Membership inference accuracy is 70% with *CALM*, which is higher than retraining at 66%. The computation time for *CALM* is 604.72 seconds, considerably faster than retraining’s 953.47 seconds. Meanwhile, the communication cost is reduced from 97.32M to 8.13M, again confirming that *CALM* retains a substantial bandwidth advantage under class-level unlearning.

On CIFAR100 with ResNet50, *CALM* yields a retained accuracy of 31%, while retraining achieves 34%. The forgetting accuracy for *CALM* is 3%, compared to 1% with retraining. *CALM* achieves a relearning accuracy of 96%, while retraining reaches 100%. The membership inference accuracy is 69% for *CALM*, higher than 61% for retraining. The computation time required by *CALM* is 783.92 seconds, significantly less than retraining’s 1140.77 seconds. The communication volume is also greatly reduced, from 97.32M to 8.46M, indicating that the communication efficiency of *CALM* is maintained consistently across data-sets.

Overall, *CALM* demonstrates competitive class-unlearning performance across all datasets, achieving strong forgetting and relearning behavior while substantially reducing computation time compared with retraining. At the same time, the retained-accuracy and MIA results indicate a measurable trade-off in utility and privacy-risk reduction. Moreover, *CALM* consistently preserves a large communication advantage because only lightweight adapters are exchanged during unlearning.

5.2.3. Sample unlearning

Table 3 summarizes the results of sample-level unlearning on CIFAR10, Caltech101, and CIFAR100 with different model architectures.

On CIFAR10 with ResNet18, *CALM* attains a retained accuracy of 52%, which, while slightly lower than retraining (56%), remains competitive. The forgetting accuracy reaches 8%, compared with 5% for retraining, and the relearning accuracy drops from 94% to 89%, indicating that *CALM* still removes a substantial portion of the target sample information but does not fully match full retraining at this fine granularity. In terms of privacy, MIA accuracy is 64%, higher than the retraining baseline of 58%, again suggesting that privacy-risk reduction

Table 3

Comparison of sample unlearning methods on CIFAR10, Caltech101 and CIFAR100. RA, FA, ReA, MIA, and Time are reported as mean $\pm$ standard deviation over five independent runs.

Dataset&Model	Method	RA $\uparrow$ (%)	FA $\downarrow$ (%)	ReA $\downarrow$ (%)	MIA $\downarrow$ (%)	Time $\downarrow$ (s)	Comm $\downarrow$ (M)
CIFAR10 ResNet18	Retrain	56 $\pm$ 0.72	5 $\pm$ 0.37	94 $\pm$ 0.53	58 $\pm$ 0.39	818.47 $\pm$ 23.6	42.73
	CALM (Ours)	52 $\pm$ 0.41	8 $\pm$ 0.39	89 $\pm$ 0.42	64 $\pm$ 0.28	529.57 $\pm$ 15.6	3.92
Caltech101 ResNet50	Retrain	52 $\pm$ 0.38	8 $\pm$ 0.49	90 $\pm$ 0.92	63 $\pm$ 0.36	1031.39 $\pm$ 42.9	97.32
	CALM (Ours)	49 $\pm$ 0.28	10 $\pm$ 0.57	83 $\pm$ 0.41	69 $\pm$ 0.46	696.42 $\pm$ 19.4	8.13
CIFAR100 ResNet50	Retrain	54 $\pm$ 0.39	7 $\pm$ 0.61	85 $\pm$ 0.47	66 $\pm$ 0.44	1281.46 $\pm$ 25.7	97.32
	CALM (Ours)	51 $\pm$ 0.36	9 $\pm$ 0.41	72 $\pm$ 0.52	71 $\pm$ 0.37	853.54 $\pm$ 28.9	8.46

at the sample level is only partial. In return, *CALM* reduces the runtime from 818.47 seconds to 529.57 seconds while also lowering communication cost from 42.73M to 3.92M.

For Caltech101 with ResNet50, *CALM* achieves a retained accuracy of 49%, close to the retraining result of 52%. The forgetting accuracy is 10%, only moderately higher than retraining's 8%, while the relearning accuracy decreases from 90% to 83%, showing that the removed sample information is harder to recover after unlearning. The MIA accuracy of *CALM* is 69%, compared with 63% for retraining, which indicates a noticeable but still incomplete reduction of membership cues. In terms of efficiency, *CALM* shortens the runtime from 1031.39 seconds to 696.42 seconds and reduces communication volume from 97.32M to 8.13M.

In the CIFAR100 with ResNet50 scenario, *CALM* maintains a retained accuracy of 51%, again close to the retraining baseline of 54%. The forgetting accuracy is 9%, compared with 7% for retraining, and the relearning accuracy drops from 85% to 72%, suggesting that the forgotten information is less likely to be recovered once removed by *CALM*. The MIA accuracy is 71%, higher than the retraining baseline of 66%, indicating that sample-level privacy-risk reduction remains more limited than in the coarser-grained settings. Meanwhile, *CALM* reduces runtime from 1281.46 seconds to 853.54 seconds and communication cost from 97.32M to 8.46M.

A consistent pattern across all three datasets is that sample-level unlearning remains more challenging than client- or class-level unlearning. Unlike client- or class-level deletion, the influence of a single sample is much more diffusely entangled with shared features and routing behavior in the trained model. As a result, adapter-level editing can only approximate the effect of fully retraining the model without that sample, which explains why *CALM* shows slightly worse forgetting accuracy and higher MIA than retraining in this setting.

Overall, *CALM* remains consistently more efficient than retraining in the sample-level setting, although its margin over retraining is smaller than in the client- and class-level cases. This pattern is consistent with the inherently higher difficulty of sample-level forgetting, where the target knowledge is more weakly localized and more tightly coupled with retained samples. Nevertheless, *CALM* still preserves a clear communication advantage across all datasets because it exchanges only lightweight adapters rather than full-model parameters. Accordingly, sample-level FU in *CALM* should be viewed as an efficient approximate solution rather than an exact replacement for retraining, and this remains an important limitation of the current method.

5.2.4. Robustness under more severe non-IID settings

To further evaluate the robustness of *CALM* under stronger client heterogeneity, we additionally conduct client-level unlearning experiments under a more severe non-IID partition with Dirichlet concentration parameter $\alpha = 0.3$. The results are summarized in Table 4.

Compared with the moderate non-IID setting ($\alpha = 1.0$), all methods exhibit degraded performance under $\alpha = 0.3$, as reflected by lower retained accuracy and generally higher forgetting, relearning, and MIA values. This trend is expected, since stronger heterogeneity increases se-

mantic drift and makes the target knowledge more unevenly distributed across clients, thereby making federated unlearning more challenging.

Despite this harder setting, *CALM* remains competitive across all three datasets. On CIFAR10 with ResNet18, *CALM* achieves 64% retained accuracy and 6% forgetting accuracy, matching the best forgetting performance while maintaining a clear communication advantage (3.92M versus 42.73M for retraining). On Caltech101 with ResNet50, *CALM* attains the highest retained accuracy among efficient non-retraining methods (44%) and the lowest communication cost (8.13M), although its forgetting accuracy is slightly worse than retraining. On CIFAR100 with ResNet50, *CALM* again preserves strong retained accuracy (41%) with substantially reduced communication overhead (8.46M), while showing a small gap to retraining in forgetting accuracy (5% vs. 4%) and a moderate gap in privacy-related metrics.

Overall, these results indicate that severe non-IID partitioning makes federated unlearning uniformly more difficult, but *CALM* still maintains a favorable forgetting-utility-efficiency trade-off and preserves its communication advantage under this more challenging setting.

5.3. Backdoor remediation

To demonstrate *CALM*'s effectiveness in addressing security threats, we conduct a dedicated experiment focusing on *backdoor attack remediation* under the FL setting. Specifically, following the classic BadNets-style attack (Gu et al., 2019) and the established methodology for backdoor-ing FL models (Bagdasaryan et al., 2020), we inject a simple visual trigger pattern (a white pixel at the bottom-right corner) into a subset of CIFAR10 images on selected malicious clients. In each experiment, 20% of the total clients are randomly designated as malicious, and their local data are partially poisoned by associating the triggered images with a fixed target label, thereby embedding a backdoor into the global model during federated training. This experimental design allows us to rigorously evaluate *CALM*'s ability to erase malicious behaviors and restore model integrity in practical FL scenarios.

We evaluate the remediation effectiveness of unlearning using the following metrics:

- **Attack success rate (ASR)** measures the proportion of trigger-carrying samples misclassified into the target label. A lower ASR after unlearning indicates more successful backdoor removal.
- **Clean accuracy (CA)** is the infected model's accuracy on clean samples without carrying the trigger. This should remain high to ensure minimal utility loss.
- **Backdoor retention score (BRS)** represents the model's accuracy on backdoor samples after the forgotten data is reintroduced and retrained. A lower BRS suggests that the backdoor is harder to relearn, indicating stronger forgetting.

Unless otherwise specified, each backdoor-remediation experiment is also repeated five times with different random seeds, and the reported ASR, CA, and BRS values are presented as mean $\pm$ standard deviation.

Table 5 compares *CALM* with Retrain (ideal upper bound), Fed-Eraser (Liu et al., 2021), Exact-Fun (E-F) (Xiong et al., 2023), and EraseClient (E-C) (Halimi et al., 2022) on both CIFAR10 + ResNet18 and CIFAR100 + ResNet50.

Table 4

Comparison of unlearning methods under a more severe non-IID partition ($\alpha = 0.3$) in the client-level unlearning setting. RA, FA, ReA, MIA, and Time are reported as mean $\pm$ standard deviation over five independent runs. Communication volume is deterministic under the given protocol and is therefore reported as a single value.

Dataset&Model	Method	RA $\uparrow$ (%)	FA $\downarrow$ (%)	ReA $\downarrow$ (%)	MIA $\downarrow$ (%)	Time $\downarrow$ (s)	Comm $\downarrow$ (M)
CIFAR10 ResNet18	Retrain	66 $\pm$ 0.41	6 $\pm$ 0.46	55 $\pm$ 0.42	59 $\pm$ 0.47	927.49 $\pm$ 24.5	42.73
	FedEraser	61 $\pm$ 0.23	8 $\pm$ 0.29	56 $\pm$ 0.31	70 $\pm$ 0.38	697.47 $\pm$ 16.1	42.73
	E-F	58 $\pm$ 0.29	8 $\pm$ 0.40	58 $\pm$ 0.41	76 $\pm$ 0.33	894.56 $\pm$ 19.7	42.73
	E-C	60 $\pm$ 0.39	7 $\pm$ 0.34	62 $\pm$ 0.34	73 $\pm$ 0.53	239.55 $\pm$ 14.6	42.73
	CALM (Ours)	64 $\pm$ 0.41	6 $\pm$ 0.32	54 $\pm$ 0.49	70 $\pm$ 0.31	511.39 $\pm$ 10.2	3.92
Caltech101 ResNet50	Retrain	48 $\pm$ 0.27	5 $\pm$ 0.32	22 $\pm$ 0.33	62 $\pm$ 0.39	1301.66 $\pm$ 21.7	97.32
	FedEraser	35 $\pm$ 0.21	8 $\pm$ 0.38	18 $\pm$ 0.41	72 $\pm$ 0.35	975.52 $\pm$ 23.5	97.32
	E-F	37 $\pm$ 0.34	7 $\pm$ 0.32	21 $\pm$ 0.47	76 $\pm$ 0.42	998.31 $\pm$ 21.5	97.32
	E-C	39 $\pm$ 0.34	9 $\pm$ 0.46	21 $\pm$ 0.39	73 $\pm$ 0.31	354.13 $\pm$ 19.6	97.32
	CALM (Ours)	44 $\pm$ 0.42	6 $\pm$ 0.36	19 $\pm$ 0.26	71 $\pm$ 0.37	693.91 $\pm$ 12.1	8.13
CIFAR100 ResNet50	Retrain	43 $\pm$ 0.37	4 $\pm$ 0.39	23 $\pm$ 0.52	62 $\pm$ 0.69	1491.51 $\pm$ 27.8	97.32
	FedEraser	33 $\pm$ 0.32	5 $\pm$ 0.31	25 $\pm$ 0.27	69 $\pm$ 0.39	1087.49 $\pm$ 17.5	97.32
	E-F	39 $\pm$ 0.57	6 $\pm$ 0.28	26 $\pm$ 0.43	76 $\pm$ 0.53	1192.53 $\pm$ 24.0	97.32
	E-C	40 $\pm$ 0.35	5 $\pm$ 0.47	23 $\pm$ 0.35	72 $\pm$ 0.43	405.73 $\pm$ 21.5	97.32
	CALM (Ours)	41 $\pm$ 0.19	5 $\pm$ 0.21	24 $\pm$ 0.24	70 $\pm$ 0.52	594.65 $\pm$ 9.3	8.46

Table 5

Backdoor attack remediation performance on CIFAR10+ResNet18 and CIFAR100+ResNet50. ASR, CA, and BRS are reported as mean $\pm$ standard deviation over five independent runs.

Dataset & Model	Method	ASR $\downarrow$ (%)	CA $\uparrow$ (%)	BRS $\downarrow$ (%)
CIFAR10 + ResNet18	Before Unlearning	93 $\pm$ 1.08	70 $\pm$ 0.42	88 $\pm$ 0.33
	Retrain	3 $\pm$ 1.52	71 $\pm$ 0.37	5 $\pm$ 0.41
	FedEraser	18 $\pm$ 0.39	69 $\pm$ 0.41	27 $\pm$ 0.56
	E-F	16 $\pm$ 0.25	66 $\pm$ 0.41	24 $\pm$ 0.29
	E-C	22 $\pm$ 0.29	67 $\pm$ 0.38	30 $\pm$ 0.27
	CALM (Ours)	8 $\pm$ 0.92	68 $\pm$ 0.54	3 $\pm$ 0.88
CIFAR100 + ResNet50	Before Unlearning	89 $\pm$ 1.36	66 $\pm$ 0.72	91 $\pm$ 0.88
	Retrain (Ideal)	7 $\pm$ 0.28	68 $\pm$ 0.36	9 $\pm$ 0.35
	FedEraser	17 $\pm$ 0.44	61 $\pm$ 0.29	30 $\pm$ 0.38
	E-F	16 $\pm$ 0.41	63 $\pm$ 0.28	28 $\pm$ 0.37
	E-C	13 $\pm$ 0.25	64 $\pm$ 0.19	33 $\pm$ 0.51
	CALM (Ours)	12 $\pm$ 0.46	64 $\pm$ 0.38	16 $\pm$ 0.29

As shown in Table 5, for CIFAR10 + ResNet18, CALM effectively reduces the ASR from 93% to 8%—closely approaching the performance of the ideal retraining strategy (3%). Meanwhile, it preserves high clean accuracy (68%), competitive with other methods, and achieves the best backdoor retention score (3%), even lower than Retrain (5%). This indicates strong resistance to backdoor relearning when poisoned data is reintroduced. In contrast, FedEraser, E-F, and E-C exhibit limited effectiveness in erasing the backdoor, yielding higher ASR and BRS. On the more challenging CIFAR100 + ResNet50 setting, all methods naturally suffer from lower clean accuracy and slightly higher residual ASR/BRS due to the increased task and model complexity. Nevertheless, CALM still substantially mitigates the backdoor: it reduces the ASR from 89% to 12%, second only to Retrain (7%), while maintaining a clean accuracy of 64%, which is close to the retraining upper bound (68%) and higher than FedEraser (61%). More importantly, CALM achieves a much lower BRS (16%) than FedEraser, E-F, and E-C (30%, 28%, and 33%, respectively), suggesting that the backdoor is significantly harder to relearn under our CLIP-assisted LoRA-MoE editing scheme.

These results validate CALM's capability in not only preserving model utility but also efficiently mitigating security threats in FL through precise and targeted unlearning.

5.4. Ablation study on components

To evaluate the contribution of each core component in CALM, we conduct a series of ablation experiments on CIFAR-10 with ResNet-18 under client-level unlearning. Specifically, we focus on two critical components: (1) the LoRA-MoE architecture for expert-specific low-rank adaptation and expert routing, and (2) the CLIP-assisted semantic consistency regularization. In CALM, LoRA provides the low-rank parameterization of each adapter expert, while MoE determines how multiple LoRA experts are selectively activated. To examine the effect of the MoE-based multi-expert design, we include a single-LoRA variant that keeps the LoRA parameterization but disables MoE routing by using only one LoRA adapter. This variant should therefore be interpreted as *single LoRA without MoE routing*, rather than as removing LoRA. We evaluate CLIP regularization separately by disabling the CLIP-assisted loss term. **LoRA-MoE.** As detailed in Table 6, replacing the LoRA-MoE module with the single-LoRA variant (second row) causes a drop in retained accuracy from 66% to 64% and an increase in forgetting accuracy from 3% to 7%, demonstrating degraded knowledge retention and less precise forgetting. The lower computation time in this variant is mainly because it keeps only one LoRA adapter and removes dynamic expert routing and expert-specific low-rank updates, thereby reducing the additional computation introduced by the LoRA-MoE editing process. However, this efficiency gain comes at the cost of weaker knowledge localization and degraded unlearning performance, highlighting the importance of the MoE-based multi-expert LoRA design for fine-grained knowledge editing.

CLIP. Disabling the CLIP-based semantic regularization (third row) leads to a decrease in forgetting performance (FA rises from 3% to 5%) and a higher membership inference accuracy (from 67% to 69%), while the retained accuracy changes only slightly. This indicates that CLIP guidance mainly improves semantic consistency during editing and helps suppress residual membership cues, rather than simply increasing retained-task accuracy. Therefore, CLIP regularization contributes to a better privacy–forgetting trade-off in the final CALM model.

Overall, the results indicate that both the MoE-based multi-expert LoRA design and CLIP contribute to the final behavior of CALM. The full model achieves the most favorable trade-off among retained utility, forgetting effectiveness, and privacy-risk reduction, while maintaining reasonable computational cost in this setting.

Table 6

Ablation study on CIFAR10-ResNet18 (Client Unlearning). RA, FA, MIA, and Time are reported as mean $\pm$ standard deviation over five independent runs.

LoRA	MoE Routing	CLIP	RA $\uparrow$ (%)	FA $\downarrow$ (%)	MIA $\downarrow$ (%)	Time $\downarrow$ (s)	Notes
✓	✓	✓	66 $\pm$ 0.39	3 $\pm$ 0.49	67 $\pm$ 0.38	473.22 $\pm$ 21.4	full
✓	✗	✓	64 $\pm$ 0.15	7 $\pm$ 0.27	70 $\pm$ 0.27	295.51 $\pm$ 13.7	single LoRA
✓	✓	✗	65 $\pm$ 0.31	5 $\pm$ 0.52	69 $\pm$ 0.42	451.67 $\pm$ 19.4	without CLIP

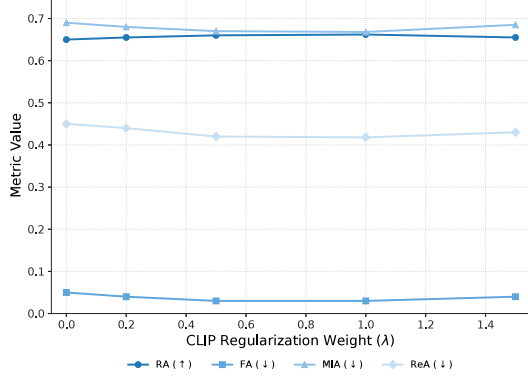


Fig. 5. Effect of CLIP regularization weight λ on unlearning performance (CIFAR10 + ResNet18).

5.5. Ablation study on hyperparameter

•CLIP regularization. We analyze the effect of the CLIP-guided semantic regularization weight λ in *CALM*, using the CIFAR10 dataset with ResNet18. As illustrated in Fig. 5, increasing λ from 0.0 to 1.0 progressively improves unlearning performance, as evidenced by reductions in forgetting accuracy (FA) and membership inference accuracy (MIA), along with a slight increase in retained accuracy (RA). The best overall performance is achieved when λ is set within the range of 0.5 to 1.0, where FA reaches its lowest value and RA attains its peak. When λ exceeds 1.0, the regularization becomes overly strong, resulting in a slight decline in RA and increased MIA and ReA. These results demonstrate that moderate CLIP regularization effectively aligns semantic representations, thereby improving both privacy protection and unlearning performance, whereas excessive regularization yields diminishing benefits.

•Client count. To assess *CALM*'s scalability, we vary the total number of clients N from 10 to 200 (Table 7). For each setting, a fixed unlearning ratio of 20% is applied, meaning that, in each experiment, 20% of the clients are randomly selected and all of their data are designated for unlearning (i.e., complete removal of their contributions from the global model). The results demonstrate that *CALM* maintains stable unlearning effectiveness as the number of clients increases, with only marginal declines in retained accuracy and privacy as N grows large. This suggests that the communication-saving benefit of adapter-only updates becomes increasingly attractive in bandwidth-constrained deployments, such as mobile and edge federated environments, where repeated full-model transmission is expensive. At the same time, these results also suggest that scaling *CALM* to larger client populations may require closer monitoring of expert routing stability and expert utilization, since a larger and more heterogeneous client pool can make expert assignment and coordination more challenging.

•LoRA-MoE expert count. We also analyze the influence of the number of LoRA-MoE expert branches, varying from 1 to 8, which results are shown in Fig. 6. Increasing the number of experts improves performance up to a point, with optimal results at four experts. Too few experts restricts specialization and forgetting precision, while too many cause over-fragmentation and reduced stability. This demonstrates that mod-

Table 7

Effect of client count N on unlearning performance (CIFAR10-ResNet18, 20% clients removed).

#Clients N	RA $\uparrow$ (%)	FA $\downarrow$ (%)	MIA $\downarrow$ (%)	ReA $\downarrow$ (%)
10	71 $\pm$ 0.33	1 $\pm$ 0.35	65 $\pm$ 0.31	40 $\pm$ 0.27
20	68 $\pm$ 0.24	2 $\pm$ 0.29	66 $\pm$ 0.14	40 $\pm$ 0.37
50	66 $\pm$ 0.39	3 $\pm$ 0.49	67 $\pm$ 0.38	42 $\pm$ 0.71
100	65 $\pm$ 0.37	4 $\pm$ 0.33	70 $\pm$ 0.29	44 $\pm$ 0.18
200	65 $\pm$ 0.43	4 $\pm$ 0.49	71 $\pm$ 0.14	44 $\pm$ 0.69

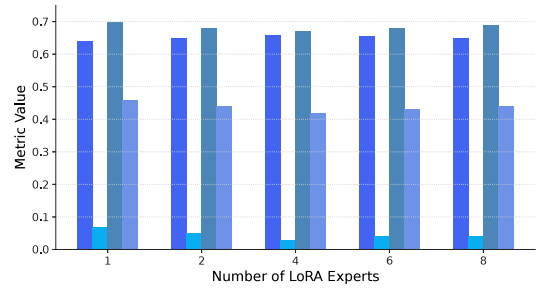


Fig. 6. Effect of LoRA-MoE expert count on unlearning performance (CIFAR10-ResNet18, 20% clients removed).

erate expert diversity is critical for maximizing the benefit of modular low-rank adaptation.

6. Conclusion

In this paper, we propose *CALM*, a unified framework for fine-grained federated unlearning that combines modular LoRA-MoE knowledge localization, CLIP-assisted semantic regularization, and targeted expert resetting. Across client-, class-, and sample-level unlearning tasks, *CALM* achieves strong forgetting-utility trade-offs while substantially reducing communication overhead through adapter-only updates. These communication savings are particularly attractive in bandwidth-constrained FL deployments, such as mobile or edge environments, where repeated full-model transmission is expensive. At the same time, our results also reveal important trade-offs. *CALM* does not uniformly minimize wall-clock time, sample-level unlearning remains more challenging than client- or class-level forgetting, and MIA remains above the ideal random-guessing level even after unlearning. We therefore interpret *CALM* as an efficient and practical approximation to retraining rather than a complete privacy guarantee. Future work will extend the framework to more severe non-IID regimes, newer backbone architectures, and stronger empirical privacy evaluation protocols.

CRedit authorship contribution statement

Mang Su: Writing – review & editing, Methodology, Conceptualization; **Yang Bai:** Writing – original draft, Software, Methodology; **Caiyun Liu:** Writing – review & editing, Validation, Supervision; **Qihao Dong:** Writing – review & editing, Formal analysis; **Hongsheng Hu:** Writing – review & editing; **Yansong Gao:** Writing – review & editing, Validation; **Anmin Fu:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., & Shmatikov, V. (2020). How to backdoor federated learning. In *International conference on artificial intelligence and statistics* (pp. 2938–2948). PMLR.
- Baruch, G., Baruch, M., & Goldberg, Y. (2019). A little is enough: Circumventing defenses for distributed learning. *Advances in Neural Information Processing Systems*, 32, 8632–8642.
- Baumhauer, T., Schöttle, P., & Zeppelzauer, M. (2022). Machine unlearning: Linear filtration for logit-based classifiers. *Machine Learning*, 111(9), 3203–3226.
- Bourtoule, L., Chandrasekaran, V., Choquette-Choo, C. A., Jia, H., Travers, A., Zhang, B., Lie, D., & Papernot, N. (2021). Machine unlearning. In *2021 IEEE symposium on security and privacy (SP)* (pp. 141–159). IEEE.
- Cao, X., Jia, J., Zhang, Z., & Gong, N. Z. (2023). Fedrecovery: Recovering from poisoning attacks in federated learning using historical information. In *2023 IEEE symposium on security and privacy (SP)* (pp. 1366–1383). IEEE.
- Che, T., Zhou, Y., Zhang, Z., Lyu, L., Liu, J., Yan, D., Dou, D., & Huan, J. (2023). Fast federated machine unlearning with nonlinear functional theory. In *International conference on machine learning* (pp. 4241–4268). PMLR.
- Chen, C., Sun, F., Zhang, M., & Ding, B. (2022a). Recommendation unlearning. In *Proceedings of the ACM web conference 2022* (pp. 2768–2777).
- Chen, K., Wang, Z., Mi, B., Liu, W., Wang, S., Ren, X., & Shen, J. (2024). Machine unlearning in large language models. arXiv:2404.16841
- Chen, M., Zhang, Z., Wang, T., Backes, M., Humbert, M., & Zhang, Y. (2022b). Graph unlearning. In *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security* (pp. 499–513).
- Chowdhury, S. B. R., Choromanski, K., Sehanobish, A., Dubey, A., & Chaturvedi, S. (2024). Towards scalable exact machine unlearning using parameter-efficient fine-tuning. arXiv:2406.16257
- Fan, C., Liu, J., Zhang, Y., Wong, E., Wei, D., & Liu, S. (2023). Salun: Empowering machine unlearning via gradient-based weight saliency in both image classification and generation. arXiv:2310.12508
- Fang, M., Cao, X., Jia, J., & Gong, N. (2020). Local model poisoning attacks to {Byzantine-Robust} federated learning. In *29th USENIX security symposium (USENIX security 20)* (pp. 1605–1622).
- Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 1–39.
- Fei-Fei, L., Fergus, R., & Perona, P. (2004). Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. In *2004 Conference on computer vision and pattern recognition workshop* (pp. 178–178). IEEE.
- Gao, Y., Kim, M., Abuadba, S., Kim, Y., Thapa, C., Kim, K., Camtepe, S. A., Kim, H., & Nepal, S. (2020). End-to-end evaluation of federated learning and split learning for internet of things. In *2020 International symposium on reliable distributed systems (SRDS)* (pp. 91–100). IEEE.
- Golatk, A., Achille, A., & Soatto, S. (2020). Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 9304–9312).
- Graves, L., Nagisetty, V., & Ganesh, V. (2021). Amnesiac machine learning. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11516–11524). (vol. 35).
- Gu, H., Zhu, G., Zhang, J., Zhao, X., Han, Y., Fan, L., & Yang, Q. (2024). Unlearning during learning: An efficient federated machine unlearning method. arXiv:2405.15474
- Gu, T., Liu, K., Dolan-Gavitt, B., & Garg, S. (2019). Badnets: Evaluating backdoor attacks on deep neural networks. *Ieee Access*, 7, 47230–47244.
- Guo, C., Goldstein, T., Hannun, A., & Van Der Maaten, L. (2019). Certified data removal from machine learning models. arXiv:1911.03030
- Halimi, A., Kadhe, S., Rawat, A., & Baracaldo, N. (2022). Federated unlearning: How to efficiently erase a client in fl? arXiv:2207.05521
- Hsu, T.-M. H., Qi, H., & Brown, M. (2019). Measuring the effects of non-identical data distribution for federated visual classification. arXiv:1909.06335
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W. et al. (2022). Lora: Low-rank adaptation of large language models. *ICLR*, 1(2), 3.
- Izzo, Z., Smart, M. A., Chaudhuri, K., & Zou, J. (2021). Approximate data deletion from machine learning models. In *International conference on artificial intelligence and statistics* (pp. 2008–2016). PMLR.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, University of Toronto.
- Li, G., Hsu, H., Chen, C.-F., & Marculescu, R. (2024). Fast-NTK: Parameter-efficient unlearning for large-scale models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 227–234).
- Li, H., Yu, L., & He, W. (2019). The impact of GDPR on global technology development. *Journal of Global Information Technology Management*, 22(1), 1–6. <https://doi.org/10.1080/1097198X.2019.1569186>.
- Liu, G., Ma, X., Yang, Y., Wang, C., & Liu, J. (2020). Federated unlearning. arXiv:2012.13891
- Liu, G., Ma, X., Yang, Y., Wang, C., & Liu, J. (2021). Federaser: Enabling efficient client-level data removal from federated learning models. In *2021 IEEE/ACM 29th international symposium on quality of service (IWQOS)* (pp. 1–10). IEEE.
- Liu, S., Yao, Y., Jia, J., Casper, S., Baracaldo, N., Hase, P., Yao, Y., Liu, C.-Y., Xu, X., Li, H. et al. (2025). Rethinking machine unlearning for large language models. *Nature Machine Intelligence*, 7, 181–194.
- Liu, Y., Xu, L., Yuan, X., Wang, C., & Li, B. (2022). The right to be forgotten in federated learning: An efficient realization with rapid retraining. In *IEEE infocom 2022-IEEE conference on computer communications* (pp. 1749–1758). IEEE.
- Liu, Z., Jiang, Y., Shen, J., Peng, M., Lam, K.-Y., Yuan, X., & Liu, X. (2024). A survey on federated unlearning: Challenges, methods, and future directions. *ACM Computing Surveys*, 57(1), 1–38.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics* (pp. 1273–1282). PMLR.
- Nguyen, T. T., Huynh, T. T., Ren, Z., Nguyen, P. L., Liew, A. W.-C., Yin, H., & Nguyen, Q. V. H. (2022). A survey of machine unlearning. arXiv:2209.02299
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J. et al. (2021). Learning transferable visual models from natural language supervision. In *International conference on machine learning* (pp. 8748–8763). PMLR.
- Shastri, S., Wasserman, M., & Chidambaram, V. (2019). The seven sins of {Personal-Data} processing systems under {GDPR}. In *11th USENIX workshop on hot topics in cloud computing (hotcloud 19)*.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv:1701.06538
- Shejwalkar, V., & Houmansadr, A. (2021). Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *NDSS*.
- Shejwalkar, V., Houmansadr, A., Kairouz, P., & Ramage, D. (2022). Back to the drawing board: A critical evaluation of poisoning attacks on production federated learning. In *2022 IEEE symposium on security and privacy (SP)* (pp. 1354–1371). IEEE.
- Tarun, A. K., Chundawat, V. S., Mandal, M., & Kankanhalli, M. (2023). Fast yet effective machine unlearning. *IEEE Transactions on Neural Networks and Learning Systems*, 35(9), 13046–13055.
- Torkzadehmahani, R., Nasirigerdeh, R., Kaissis, G., Rueckert, D., Dziugaite, G. K., & Triantafillou, E. (2024). Improved localized machine unlearning through the lens of memorization. arXiv:2412.02432
- Wang, J., Guo, S., Xie, X., & Qi, H. (2022). Federated unlearning via class-discriminative pruning. In *Proceedings of the ACM web conference 2022* (pp. 622–632).
- Wang, W., Tian, Z., Zhang, C., Liu, A., & Yu, S. (2023). BFU: Bayesian federated unlearning with parameter self-sharing. In *Proceedings of the 2023 ACM Asia conference on computer and communications security* (pp. 567–578).
- Warnecke, A., Pirsch, L., Wressnegger, C., & Rieck, K. (2021). Machine unlearning of features and labels. arXiv:2108.11577
- Wu, C., Zhu, S., & Mitra, P. (2022a). Federated unlearning with knowledge distillation. arXiv:2201.09441
- Wu, L., Guo, S., Wang, J., Hong, Z., Zhang, J., & Ding, Y. (2022b). Federated unlearning: Guarantee the right of clients to forget. *IEEE Network*, 36(5), 129–135.
- Xie, C., Huang, K., Chen, P.-Y., & Li, B. (2019). DBA: Distributed backdoor attacks against federated learning. In *International conference on learning representations*.
- Xiong, Z., Li, W., Li, Y., & Cai, Z. (2023). Exact-fun: An exact and efficient federated unlearning approach. In *2023 IEEE international conference on data mining (ICDM)* (pp. 1439–1444). IEEE.
- Zhang, P.-F., Bai, G., Huang, Z., & Xu, X.-S. (2022a). Machine unlearning for image retrieval: A generative scrubbing approach. In *Proceedings of the 30th ACM international conference on multimedia* (pp. 237–245).
- Zhang, Z., Cao, X., Jia, J., & Gong, N. Z. (2022b). Fldetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 2545–2555).